

Fix for `type_order` template definition

Document #: P3778R0 [[Latest](#)] [[Status](#)]
Date: 2025-07-03
Project: Programming Language C++
Audience: LEWG, LWG
Reply-to: Gašper Ažman
<gasper.azman@gmail.com>

Contents

- [1 Abstract and problem statement](#)
- [2 Design and discussion](#)
- [3 Proposal](#)
- [4 Notes](#)
- [5 Acknowledgements](#)
- [6 References](#)

§ 1 Abstract and problem statement

The `type_order` template is unimplementable as specified by [\[P2830R10\]](#), due to values of `type_ordering` not being usable as template parameters because `strong_ordering` is not a structural type (it has a private member).

We propose replacing the *Cpp17BinaryTypeTrait* with a *base characteristic* wording choice by an explicitly spelled out the struct body to avoid the issue while preserving all possible uses.

§ 2 Design and discussion

The problematic part of [\[P2830R10\]](#) wording is as follows:

- 2 The name `type_order` denotes a *Cpp17BinaryTypeTrait* ([\[meta.rqmts\]](#)) with a base characteristic of `integral_constant<strong_ordering, TYPE-ORDER(X, Y)>`.

This *Cpp17BinaryTypeTrait* concept is defined to inherit from `integral_constant<value-type, trait-value>`. Unfortunately, this is not possible if the intended value type is

not a structural type.

`integral_constant` is defined in [\[meta.help\]](#) like so:

```
template<class T, T v> struct integral_constant {
    static constexpr T value = v;

    using value_type = T;
    using type = integral_constant<T, v>;

    constexpr operator value_type() const noexcept { return value; }
    constexpr value_type operator()() const noexcept { return value; }
};
```

The specific intended uses for the design of *Cpp17BinaryTypeTrait* are:

1. Implicit type conversion to `integral_constant<T, v>` *to hide the arguments to the type trait* and thus potentially instantiate fewer templates if used deliberately (**cannot preserve**).
2. The embedded `::type` member to aid doing (1) explicitly (**cannot preserve**).
3. The value member to serve as the type trait result.
4. The `value_type` member to get the type of value.
5. `constexpr` implicit conversion operator to value for convenience.
6. `constexpr` nullary call operator to help accessing value from contexts that expect an invocable function.

We should preserve all properties we can: (3), (4), (5), and (6).

The behaviour should be as close to the rest of type traits as possible, so we should preserve the exact behaviour of `integral_constant` in every possible way.

We are left with the following design:

```
template<class T, class U> struct type_order {
    static constexpr strong_ordering value = TYPE-ORDER(T, U);

    using value_type = strong_ordering;

    constexpr operator value_type() const noexcept { return value; }
    constexpr value_type operator()() const noexcept { return value; }
};
```

§ 3 Proposal

In [\[compare.type\]](#), strike paragraph 2, and fill out the code snippet in paragraph 1:

- 1 There is an implementation-defined total ordering of all types. For any (possibly incomplete) types X and Y , the expression `TYPE-ORDER( $X$ ,  $Y$ )` is a constant

expression (`[expr.const]`) of type `strong_ordering` (`[cmp.strongord]`). Its value is `strong_ordering::less` if X precedes Y in this implementation-defined total order, `strong_ordering::greater` if Y precedes X , and `strong_ordering::equal` if they are the same type.

[Note 1: `int`, `const int` and `int&` are different types. – end note]

[Note 2: This ordering need not be consistent with the one induced by `type_info::before`. – end note]

[Note 3: The ordering of TU-local types from different translation units is not observable, because the necessary specialization of `type_order` is impossible to name. – end note]

```
template <class T, class U>
struct type_order;

template<class T, class U> struct type_order {
    static constexpr strong_ordering value = TYPE-ORDER(T, U);

    using value_type = strong_ordering;

    constexpr operator value_type() const noexcept { return value; }
    constexpr value_type operator()() const noexcept { return value; }
};
```

- 2 The name `type_order` denotes a *Cpp17BinaryTypeTrait* (`[meta.rqmts]`) with a base characteristic of `integral_constant<strong_ordering, TYPE-ORDER(X, Y)>`.

§ 4 Notes

As of the time of this writing, `libstdc++`'s trunk implementation is as follows:

```
template<typename _Tp, typename _Up>
struct type_order
{
    static constexpr strong_ordering value = __builtin_type_order(_Tp,
        _Up);
    using value_type = strong_ordering;
    using type = type_order<_Tp, _Up>;
    constexpr operator value_type() const noexcept { return value; }
    constexpr value_type operator()() const noexcept { return value; }
};
```

Note that it has type.

§ 5 Acknowledgements

The author would like to thank the following people for their contributions:

- Jakub Jelinek for implementing it in GCC and raising the alarm
- Jonathan Wakely for making sure I heard the alarm and explaining what it's about
- Barry Revzin for spelling out the obvious design above.
- Tim Song for pointing out the reason the type member is in `integral_constant` and that including it does us a disservice.
- Bronek Kozicki for wrangling the design space and discussion

§ 6 References

[P2830R10] Gašper Ažman, Nathan Nichols. 2025-03-15. Standardized Constexpr Type Ordering.

<https://wg21.link/p2830r10>